DESIGN AND IMPLEMENTATION OF A MULTI-TENANT SAAS PLATFORM FOR BUSINESS INVENTORY MANAGEMENT USING MICROSERVICES ARCHITECTURE

A Project Proposal

Submitted in Partial Fulfilment of the Requirements for the Award of the Higher National Diploma (HND) in Software Engineering

Department of Computer Science / Software Engineering

2026

Presentation Outline

This proposal is organised under the following sections:

1. Background of the Study
2. Review of Related Literature
3. Motivation
4. Aim and Objectives
5. Methodology
6. Expected Results
7. Conclusion
8. References

1.0 Background of the Study

- Multi-tenancy is an architectural model in which a single software instance serves multiple customer organisations while keeping each tenant's data and configuration logically separated (Kavis, 2014).
- Small and medium-sized businesses that rely on single-tenant or spreadsheet-based inventory systems often face high maintenance costs and limited scalability as their operations grow (Mietzner et al., 2009).
- Microservices architecture decomposes a large application into small, independently deployable services that can be developed, scaled and maintained on their own (Tizzei et al., 2017).
- Industrial case studies have shown that combining multi-tenancy with microservices improves scalability, eases third-party integration, and allows new tenants to be onboarded without re-architecting the whole system (Batista et al., 2022).
- Despite these benefits, achieving secure data isolation, fair resource allocation and consistent performance for tenants that share the same infrastructure remains a significant engineering challenge (Bezemer et al., 2010).
- Many Nigerian retail and distribution businesses still track stock manually or on spreadsheets, which leads to stock discrepancies, late reordering and avoidable revenue loss — a gap that a cloud-based multi-tenant inventory platform can help close.

2.0 Review of Related Literature

The table below summarises five key studies that informed the design direction of this project.

Title	Author + Year	Objectives	Methodology	Results	Limitations
Using Microservices and SPL Engineering to Support Reuse of Evolving Multi-Tenant SaaS	Tizzei et al. (2017)	Investigate combining microservices with software product line	Empirical study tracking code reuse across tenant	Average software reuse of 62% of lines of code	Findings drawn from a single industrial system; generalisability untested.

Title	Author + Year	Objectives	Methodology	Results	Limitations
		engineering for multi-tenant SaaS reuse.	deployments over time.	achieved across tenants.	
Towards a Multi-Tenant Microservice Architecture: An Industrial Experience	Batista et al. (2022)	Examine how multi-tenancy in a microservice architecture affects scalability and integration.	Migration of a legacy monolith to microservices, with a scalability evaluation.	Improved scalability, easier management and smoother third-party integration.	Evaluation limited to one company's migration context.
Variability Modeling to Support Customization and Deployment of Multi-Tenant-Aware SaaS Applications	Mietzner et al. (2009)	Model variability so SaaS providers can customise deployments per tenant.	Conceptual variability modelling combined with deployment automation.	Reduced effort in configuring and deploying tenant-specific variants.	Theoretical model with limited large-scale empirical validation.
Enabling Multi-Tenancy: An Industrial Experience Report	Bezemer et al. (2010)	Report practical lessons from converting a single-tenant application to multi-tenant.	Industrial case study and source-code level analysis of the conversion.	Identified recurring code-level patterns that complicate multi-tenant conversion.	Findings are specific to the studied legacy codebase.
Architecting the Cloud: Design Decisions for Cloud Computing Service Models	Kavis (2014)	Provide design guidance for building SaaS, PaaS and IaaS solutions.	Practitioner-oriented synthesis of cloud architecture design decisions.	Established widely cited design principles for multi-tenant cloud systems.	Largely prescriptive; lacks original empirical performance data.

3.0 Motivation

- Most SaaS inventory tools available to Nigerian SMEs are single-tenant, expensive to host individually, and difficult for a vendor to maintain at scale.
- A microservices-based, multi-tenant design lets one platform serve many businesses from shared infrastructure, lowering the cost per tenant while keeping data isolated.
- Independently deployable services make it possible to update, scale, or fix one part of the system (e.g. stock alerts) without taking the whole platform offline for every tenant.
- Building this project demonstrates practical mastery of modern cloud-native engineering practices — service decomposition, API gateways, and tenant-aware data design — that are in high demand in the software industry.

4.0 Aim and Objectives

Aim: To design and implement a multi-tenant Software-as-a-Service (SaaS) platform for business inventory management, built on a microservices architecture, that allows multiple independent businesses to manage stock, sales and reorder activity securely from a single shared system.

Specific Objectives

1. Review existing literature on multi-tenant SaaS design and microservices architecture to establish sound design principles.
2. Design a tenant-aware microservices architecture covering authentication, inventory, order and notification services.
3. Implement core services with isolated tenant data using a shared-database, separate-schema strategy.
4. Develop a REST API gateway that routes and authenticates requests across services for each tenant.
5. Evaluate the resulting system against scalability, response-time and data-isolation metrics.

5.0 Methodology

5.1 Data Collection Methods

6. Literature Review: Existing journal articles, conference papers and industry reports on multi-tenant SaaS and microservices were reviewed to ground design decisions in established practice.
7. Structured Interviews: Informal stock managers and small-business owners were interviewed to understand current inventory pain points, reorder habits and reporting needs.
8. Document Analysis: Sample stock sheets, sales registers and existing inventory templates were examined to extract realistic data fields and business rules for the system design.

5.2 Proposed System Architecture

The platform follows a layered, tenant-aware microservices architecture. Each layer is summarised below:

Layer	Description
Client Layer	Tenant web and mobile clients (Business A, Business B, Business C, ...) send requests to the platform over HTTPS.
API Gateway	A single entry point that authenticates each request, resolves the tenant identity from the request token, and routes the call to the correct downstream microservice.
Microservices Layer	Independently deployable services — Authentication & Tenant Management, Inventory Service, Order & Sales Service, and Notification Service — each owning its own logic and able to scale on its own.
Data Layer	A shared-database, separate-schema-per-tenant strategy keeps each tenant's inventory and sales records isolated while controlling infrastructure cost.
Messaging Layer	An asynchronous message broker (e.g. RabbitMQ) carries cross-service events, such as low-stock alerts, between services without tightly coupling them.

In summary, tenant clients reach the platform through a single API Gateway, which authenticates the request and resolves the tenant identity before routing it onward. Independent microservices handle inventory, orders, authentication and notifications, each scalable on its own. Tenant data is kept isolated using a separate schema per tenant within a shared database, balancing

cost-efficiency with strong data separation, while a message broker carries cross-service events such as low-stock alerts.

5.3 Software Development Model

This project adopts the Agile (Scrum) model, delivered as a repeating sprint cycle made up of the following phases:

Phase	Description
Requirements	Gather and document functional and non-functional requirements for the current sprint's target service or feature.
Design	Produce a lightweight design for the service or feature, including API contracts and data model changes.
Develop	Implement the feature within the sprint, following coding standards and committing incrementally to version control.
Test	Run unit and integration tests against the new or changed service before it is merged.
Deploy	Containerise and deploy the updated service independently of the others, using Docker.
Review	Hold a sprint review and retrospective to gather feedback and refine the backlog for the next sprint.

Key Features

- Work is delivered in short, fixed-length sprints, allowing the design to adapt as feedback on tenant needs comes in.
- Each sprint produces a working increment (e.g. one microservice), which fits naturally with independently built services.
- Continuous review and retrospectives let the system evolve as new requirements surface during implementation.

Reasons for Adoption

- Microservices are best built incrementally, one service at a time, matching Scrum's sprint-based delivery.

- Frequent stakeholder feedback reduces the risk of building features tenants do not actually need.

5.4 System Evaluation Metrics

9. Response Time / Latency: Average time taken for a request (e.g. stock update or order creation) to complete, measured under varying numbers of simulated concurrent tenants.
10. Tenant Scalability: The number of tenants and concurrent users the platform can serve while keeping response time and error rate within acceptable bounds.
11. Data Isolation Integrity: Verification that no tenant can access or modify another tenant's inventory records, tested through controlled cross-tenant access attempts.

5.5 Tools and Implementation Environment

- Node.js (NestJS) / Spring Boot — Backend frameworks used to build each independently deployable microservice with clean, modular structure.
- React.js — Front-end library for building the tenant-facing inventory dashboard and admin interface.
- PostgreSQL — Relational database engine used with a separate schema per tenant for structured, isolated inventory data.
- Docker & Docker Compose — Containerises each microservice so it can be built, shipped and run consistently across environments.
- Kong / NGINX (API Gateway) — Routes incoming requests to the correct microservice and resolves tenant identity before forwarding.
- RabbitMQ — Message broker used for asynchronous communication between services, such as stock-alert events.
- Postman & Jest — Tools for testing REST API endpoints and writing automated unit/integration tests.
- Git & GitHub — Version control and collaboration platform for tracking source-code changes throughout development.

6.0 Expected Results

On completion, the system is expected to be capable of the following:

- Multi-Tenant Onboarding: Multiple independent businesses will be able to register and operate on the same platform with fully isolated inventory data.
- Real-Time Stock Tracking: Tenants will be able to add, update and monitor stock levels in real time, with automatic low-stock alerts.
- Independent Service Scaling: Individual microservices (e.g. inventory or orders) will scale independently based on demand, without affecting other services.
- Sales & Inventory Reports: The system will generate sales summaries and stock reports that help tenants make reorder and pricing decisions.
- Secure Tenant Isolation: Each tenant's data will remain inaccessible to other tenants, verified through access-control testing.
- Acceptable Performance Under Load: The platform will maintain acceptable response times as the number of simulated concurrent tenants increases.

7.0 Conclusion

This project will design and implement a multi-tenant SaaS platform for business inventory management built on a microservices architecture. By giving multiple independent businesses secure, isolated access to a single, scalable system, the platform addresses the cost, scalability and maintenance challenges faced by existing single-tenant inventory solutions. The resulting system is expected to demonstrate that microservices and multi-tenancy can be combined to deliver a practical, affordable inventory management solution suited to small and medium-sized businesses.

8.0 References

Batista, C., Proença, B., Cavalcante, E., Batista, T., Morais, F., & Medeiros, H. (2022). Towards a multi-tenant microservice architecture: An industrial experience. In Proceedings of the

46th IEEE Annual Computers, Software, and Applications Conference (pp. 553–562). IEEE.

Batista, C., Cavalcante, W. B. R., Proença, B., Batista, T., & Morais, F. (2024). Managing asynchronous workloads in a multi-tenant microservice enterprise environment. Software: Practice and Experience, 54(2), 334–359.

Bezemer, C.-P., Zaidman, A., Platzbeecker, B., Hurkmans, T., & Hart, A. (2010). Enabling multi-tenancy: An industrial experience report. In Proceedings of the 2010 IEEE International Conference on Software Maintenance (pp. 1–8). IEEE.

Kavis, M. J. (2014). Architecting the cloud: Design decisions for cloud computing service models (SaaS, PaaS, and IaaS). John Wiley & Sons.

Mietzner, R., Metzger, A., Leymann, F., & Pohl, K. (2009). Variability modeling to support customization and deployment of multi-tenant-aware software as a service applications. In Proceedings of the ICSE Workshop on Principles of Engineering Service-Oriented Systems (pp. 18–25). IEEE Computer Society.

Newman, S. (2021). Building microservices: Designing fine-grained systems (2nd ed.). O'Reilly Media.

Tizzei, L. P., Nery, M., Segura, V. C. V. B., & Cerqueira, R. F. G. (2017). Using microservices and software product line engineering to support reuse of evolving multi-tenant SaaS. In Proceedings of the 21st International Systems and Software Product Line Conference – Volume A (pp. 205–214). ACM.